FineLine Cinema

Software Requirements Specification

Version 1.2

Date 03/19/2026

Group #17
Esteban Naranjo, Jackson Adams, Bryan Ly

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Spring 2026

Revision History

Date	Description	Author	Comments
Feb 19, 2026	Version 1.0	Esteban Naranjo, Jackson Adams, Bryan Ly	Initial SRS Submission
Mar 3, 2026	Version 1.1	Esteban Naranjo, Jackson Adams, Bryan Ly	Design Specification SRS Submission
Mar 19, 2026	Version 1.2	Jackson Adams, Esteban Naranjo, Bryan Ly	Test Case Specification SRS Submission

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
E.J.B.	Esteban, Jackson, Brian	Software Eng.	Feb 27, 2026
	Dr. Gus Hanna	Instructor, CS 250	
E.J.	Esteban, Jackson	Software Eng.	Mar 3, 2026
E.J.B.	Esteban, Jackson, Brian	Software Eng.	Mar 19, 2026

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 2
1.4 References 3
1.5 Overview 3

2. General Description 3

2.1 Product Perspective 4
2.2 Product Functions 4
2.3 User Characteristics 4
2.4 General Constraints 5
2.5 Assumptions and Dependencies 5

3. Specific Requirements 6

3.1 External Interface Requirements 6
3.1.1 User Interfaces *6*
3.1.2 Hardware Interfaces *7*
3.1.3 Software Interfaces *8*
3.1.4 Communications Interfaces *8*
3.2 Functional Requirements 8
3.2.1 <Functional Requirement or Feature #1> *8*
3.2.2 <Functional Requirement or Feature #2> *8*
3.3 Use Cases *12*
3.3.1 Use Case #1 ..... *12*
3.3.2 Use Case #2 ..... *12*
3.4 Classes / Objects 14
3.4.1 <Class / Object #1> *14*
3.4.2 <Class / Object #2> *15*
3.4.3 <Class / Object #3> *15*
3.4.4 <Class / Object #4> *16*
3.4.5 <Class / Object #5> *16*
3.4.6 <Class / Object #6> *16*
3.4.7 <Class / Object #7> *17*
3.4.8 <Class / Object #8> *17*
3.5 Non-Functional Requirements 18
3.5.1 Performance *18*
3.5.2 Reliability *18*
3.5.3 Availability *18*
3.5.4 Security *19*
3.5.5 Maintainability *20*
3.5.6 Portability *20*
3.6 Brife Overview of System 21
3.7 Design Constraints 22
3.8 Logical Database Requirements 22
3.9 Other Requirements 22

4. Software Architecture Overview 22

4.1 Major Components 23

4.2 Software Architecture Diagram..... 23
4.3 UML Class Description..... 24
4.4 UML Class Diagram..... 30

5. Development Plan and Task Partitioning..... 31

5.1 System Implementation Breakdown..... 31
5.2 Development Timeline..... 32
5.3 Team Member Responsibilities..... 33

6. Testing..... 34

6.1 Testing Plan..... 34
6.2 Test Cases..... 35
6.3 Unit Testing..... 38
6.4 Integration Testing..... 41
6.5 System Testings..... 43
6.6 Test Case Documentation..... 45

1. Introduction

This Software Requirements Specification (SRS) describes the requirements for the FineLine Cinema Ticketing System. It defines the functional and non-functional requirements needed for the design, development, testing, and maintenance of the system.

This document is intended for the development team, project stakeholders, and theater staff who will use or manage the system. It provides a complete description of what the system must do and the constraints under which it must operate.

1.1 Purpose

The purpose of this document is to provide thorough and complete information about the requirements of the FineLine Cinema Ticketing System. This document outlines the system's requirements and the constraints under which it will operate.

The intended audience includes the development team and the cinema staff, who will use and manage the system. This document will guide the design, implementation, and testing of the system.

1.2 Scope

The software product to be developed is the FineLine Cinema Ticketing System, a web-based ticketing platform for movie theaters located in the San Diego area.

The system will allow customers to browse movies, view trailers, select theaters and showtimes, purchase tickets, select seats (for deluxe auditoriums), and order food and drinks. The system will also allow theater administrators to manage movie listings, showtimes, pricing, seating availability, and ticket data. The system will support both online and in-person ticket sales through a shared real-time database.

The system will initially serve 20 theaters with 10 auditoriums per theater operating within the Pacific time zone. Each regular auditorium contains 150 seats, and each deluxe auditorium contains 75 assigned seats. Customers may purchase up to 20 tickets per transaction and may book tickets up to two weeks in advance.

The constraints for the system are such that the system will not process refunds online, as refunds will be handled in person by theater staff. The system will not include a standalone mobile application and will operate through standard web browsers.

The primary objectives of this system are to:

- Improve the efficiency of ticket purchasing and seat management
- Reduce existing performance and reliability issues in the current system
- Support up to 24,000 concurrent users during peak demand periods
- Provide real-time updates of seat availability across all theaters
- Offer a secure and user-friendly experience for customers and staff

This document defines the complete set of functional and non-functional requirements necessary to design, implement, and test the FineLine Cinema Ticketing System.

1.3 Definitions, Acronyms, and Abbreviations

SRS – Software Requirements Specification. A document that defines the functional and non-functional requirements of a software system.

FineLine Cinema Ticketing System – The web-based ticketing platform described in this document.

Administrator – A theater staff member responsible for managing movies, showtimes, pricing, seating, and system data.

Customer – A user who accesses the system to browse movies, purchase tickets, or order food and drinks.

Deluxe Theater – A theater with assigned seating and 75 seats.

Regular Theater – A theater with general seating and 150 seats.

Checkout Timer – The five-minute period during which selected tickets or seats are reserved before being released.

Concurrent Users – The number of users accessing the system at the same time.

Membership Account – A subscription-based account that allows customers to reserve tickets under membership benefits.

HTTPS – HyperText Transfer Protocol Secure. The encrypted communication protocol is used between the client browser and the server.

Third-Party Payment Service – An external payment processing provider used to securely process credit or debit card transactions.

1.4 References

San Diego State University. (2026). CS250 Software Requirements Specification (SRS) template.

San Diego State University. (2026). Lecture 3 – Requirements gathering [PowerPoint slides].

IEEE Computer Society. (1998). IEEE 830-1998: IEEE guide to software requirements specifications. IEEE.

AMC Theatres. (n.d.). AMC Chula Vista 10 showtimes.
https://www.amctheatres.com/movie-theatres/san-diego/amc-chula-vista-10/showtimes

1.5 Overview

This Software Requirements Specification (SRS) is organized into three main sections.

Section 1 provides an introduction to the document, including its purpose, scope, definitions, references, and overall organization.

Section 2 describes the general characteristics of the FineLine Cinema Ticketing System, including the product perspective, major system functions, user characteristics, constraints, and assumptions.

Section 3 presents the specific requirements of the system. This includes external interface requirements, functional requirements, use cases, classes and objects, and non-functional requirements such as performance, reliability, availability, security, maintainability, and portability.

Together, these sections define the complete set of requirements needed for the design, development, and testing of the FineLine Cinema Ticketing System.

2. General Description

This section gives a general overview of the FineLine Cinema Ticketing System and the factors that affect how the system will be built. It explains who the system is for, what it is meant to do, and the basic assumptions and limits that shape it. The system is designed to let customers buy movie tickets online using common payment methods such as credit or debit cards, and to be usable by a wide range of customers, including support for multiple languages if needed. This section does not go into detailed system rules, but instead provides background information to help understand the specific requirements described later in this document.

2.1 Product Perspective

The FineLine Cinema Ticketing System is a software system designed to support the sale and management of movie tickets for a single movie theater or theater chain. Instead of relying only on in-person ticket sales, the system allows customers to view movie trailers, showtimes, select seats, and purchase tickets online. The system also allows customers to order food that will be delivered to their seats during the show, with the option to continue adding food during the movie and have the charges applied to their account. In addition, the system provides theater staff with tools to manage movies, showtimes, and seat availability. Overall, the ticketing system supports the theater's day-to-day operations by connecting customers, staff, and payment services within a single system.

2.2 Product Functions

The FineLine Cinema Ticketing System will provide the following main functions:

- Allow customers to browse movies that are now playing and coming soon
- Allow customers to watch movie trailers
- Allow customers to search for movies by title, genre, or showtime
- Allow customers to select a theater location (optionally using device location services)
- Allow customers to select showtimes and reserve seats
- Allow customers to purchase tickets online
- Allow customers to order food and drinks in advance or during the movie
- Allow customers to view promotions, rewards, or membership options
- Allow customers to create and manage an account
- Allow theater staff to manage movie listings, showtimes, and seat availability
- Provide an interactive seat map for assigned-seating theaters
- Display seat availability status (available, selected, occupied)
- Identify wheelchair and companion seating
- Process electronic payments securely
- Send confirmation notifications for ticket and food purchases
- Provide digital tickets that can be scanned at the theater
- Provide refund capabilities (in person)

2.3 User Characteristics

The primary users of the system are customers and theater administrators. Customers are general users with different levels of technical experience who want to browse movies, select showtimes, and purchase tickets. Because customers may not be highly technical, the system should be simple to use and easy to navigate.

Theater administrators are staff members responsible for managing movies, showtimes, seating availability, and basic system updates. Administrators are expected to have basic computer skills and a general understanding of theater operations.

2.4 General Constraints

The following general constraints apply to the FineLine Cinema Ticketing System:

The FineLine Cinema Ticketing System must operate as a web-based system and be accessible through standard internet browsers. The system must support both online ticket purchases and in-person sales, with both methods sharing the same live data and database.

The system will initially serve 20 theaters with 10 auditoriums each, located in the San Diego area, and operate within the Pacific time zone. Each regular auditorium contains 150 seats, while deluxe auditoriums contain 75 assigned seats. The system must support a minimum of 24,000 concurrent users during peak operational periods.

Ticket purchases are limited to a maximum of 20 tickets per transaction. Customers may book tickets up to two weeks in advance and may purchase tickets up to 10 minutes after showtime.

The project budget is approximately $500,000, and the client expects a viable prototype within four months.

These constraints may affect how the system is designed and implemented.

2.5 Assumptions and Dependencies

The development of the FineLine Cinema Ticketing System is based on several key assumptions and dependencies:

It is assumed that users will have access to a reliable internet connection and a supported web browser in order to use the system. The system also assumes that a centralized database will store information for all theaters, organized by location.

The system depends on secure third-party payment services to process electronic transactions. It also assumes that theater administrators will maintain accurate and up-to-date movie schedules, pricing information, and seat availability.

Refunds will be handled in person by theater staff and will not be processed directly through the system.

Some features, such as recommending nearby theaters using location services, may be considered in future versions of the system.

If any of these assumptions change, the requirements described in this document may need to be updated.

3. Specific Requirements

This section lists the detailed requirements for the FineLine Cinema Ticketing System. These requirements are written to guide the system's design, implementation, and testing. Each requirement is stated clearly so it can be checked and verified. Requirements in this section are numbered to make them easy to reference in later documents (use cases, testing, and future updates). This section focuses on what the system must do, not how it will be built.

3.1 External Interface Requirements

This section defines how the FineLine Cinema Ticketing System interacts with users, hardware, external software, and communication networks.

3.1.1 User Interfaces

UI-1 (High Priority)
The system shall be accessible through standard modern web browsers.

UI-2 (High Priority)
The system shall allow customers to browse movies currently playing and movies available within two weeks.

UI-3 (High Priority)
The system shall display available showtimes for each movie.

UI-4 (High Priority)
The system shall allow customers to select the number of tickets (a maximum of 20 per transaction).

UI-5 (High Priority)
For deluxe auditoriums, the system shall display a visual seating chart showing available, selected, and occupied seats.

UI-6 (High Priority)
The system shall visually distinguish wheelchair accessible seats and companion seats on the seating chart.

UI-7 (High Priority)
The system shall allow customers to select seats in deluxe auditoriums.

UI-8 (Medium Priority)
For regular auditoriums, the system shall allow ticket purchase without assigned seating.

UI-9 (High Priority)
The system shall provide a five minute timer once the customer begins the checkout process.

UI-10 (High Priority)
The system shall allow customers to enter payment information and complete a purchase.

UI-11 (Medium Priority)
The system shall allow customers to purchase food and drinks during checkout.

UI-12 (Medium Priority)
The system shall allow customers to add additional food and drinks during the movie, which will be charged to their account.

UI-13 (Low Priority)
The system shall display movie ratings and review scores (e.g., Rotten Tomatoes or IMDb scores).

UI-14 (Medium Priority)
The system shall display a feedback option (e.g., smiley face rating) after purchase.

UI-15 (High Priority)
The system shall provide an administrator interface for managing movies, showtimes, pricing, and seat availability.

3.1.2 Hardware Interfaces

HW-1 (High Priority)
The system shall operate on customer devices including desktop computers, laptops, and tablets.

HW-2 (High Priority)
The system shall support a centralized database server used by all 20 theaters.

HW-3 (Medium Priority)
The system shall share ticket information with the theater's in person sales system to keep seat availability updated in real time.

3.1.3 Software Interfaces

SW-1 (High Priority)
The system shall interface with secure third party payment processing services.

SW-2 (High Priority)
The system shall connect to a centralized database partitioned by theater location.

SW-3 (Medium Priority)
The system shall support integration with membership accounts for subscription based reservations.

SW-4 (Low Priority)
The system shall interface with external movie rating services to retrieve review scores.

3.1.4 Communications Interfaces

COM-1 (High Priority)
The system shall operate over standard internet protocols (HTTPS).

COM-2 (High Priority)
All communication between the client browser and server shall be encrypted.

COM-3 (High Priority)
The system shall support at least 24,000 concurrent users while maintaining consistent response times, transaction reliability, and overall system stability.

COM-4 (Medium Priority)
The system shall provide real-time updates to seat availability across online and in-person sales systems.

3.2 Functional Requirements

This section describes the specific functional features of the FineLine Cinema Ticketing System. Each feature is defined in terms of inputs, processing, outputs, and error handling.

3.2.1 Ticket Purchase Management #1

3.2.1.1 Introduction
This feature allows customers to select a movie, choose a showtime, select the number of tickets (up to 20), and complete a secure purchase through the web based system.

This feature is critical to the primary function of the system.

3.2.1.2 Inputs

- Selected movie
- Selected theater location
- Selected showtime
- Number of tickets (1–20)
- Ticket type (regular, student, military, senior)
- Payment information (credit/debit card)
- Membership ID (if applicable)

3.2.1.3 Processing

The system shall:

- Verify ticket availability
- Validate the ticket quantity does not exceed 20 per transaction
- Apply any applicable discounts
- Start a five minute checkout timer
- Process payment through third party payment service
- Confirm transaction success or failure
- Update seat/ticket availability in real time

3.2.1.4 Outputs

- Digital ticket confirmation
- Order receipt
- Confirmation email/notification
- Updated seat availability in system

3.2.1.5 Error Handling

The system shall:

- Display an error message if ticket quantity exceeds 20
- Display an error message if payment fails
- Release selected tickets if checkout timer expires
- Prevent purchase if showtime exceeds allowed purchase window

3.2.2 Seat Selection Management #2

3.2.2.1 Introduction

This feature allows customers to view seat availability in real time and select seats for the deluxe auditorium.

3.2.2.2 Inputs

- Selected theater
- Selected movie
- Selected showtime
- Seat selection (for deluxe auditorium)

3.2.2.3 Processing

The system shall:

- Display interactive seat map
- Show seat status (available, selected, occupied)
- Mark's wheelchair is accessible, and has companion seats
- Prevent the selection of occupied seats
- Reserve selected seats during checkout timer

3.2.2.4 Outputs

- Visual confirmation of selected seats
- Reserved seat status during checkout
- Updated seat availability after successful purchase

3.2.2.5 Error Handling

The system shall:

- Prevent selection of unavailable seats.
- Display a message if a seat becomes unavailable during checkout.
- Release reserved seats if the checkout timer expires before payment confirmation.

3.2.3 Food Order

3.2.3.1 Introduction

This feature allows customers to order food ahead of time and during the movie until 30 minutes before the movie's end time.

3.2.3.2 Inputs

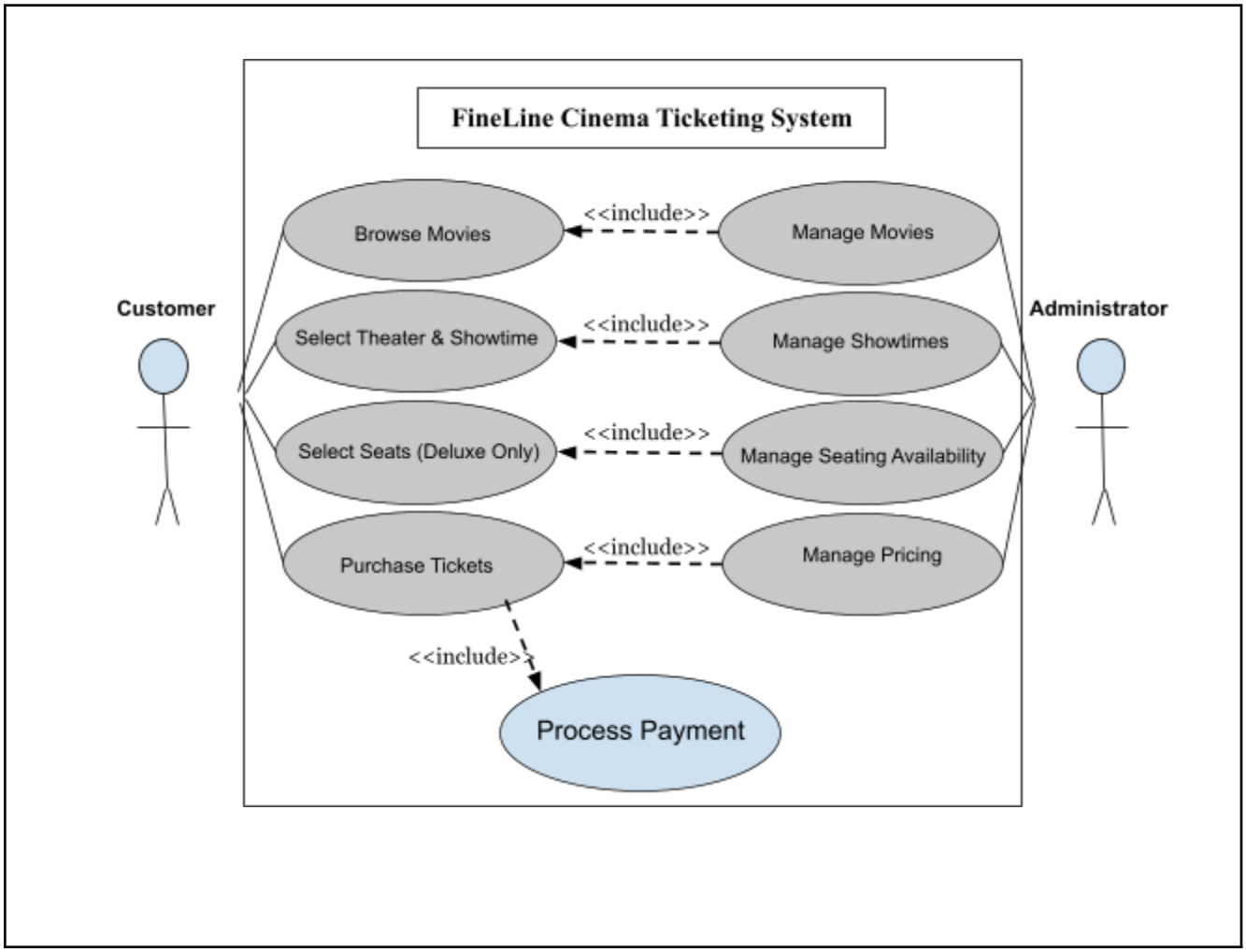
- Selected Theater
- Selected Auditorium
- Selected Movie
- Selected Showtime
- Selected Seat
- Ticket Confirmation
- Selected Items
- Payment Method

3.2.3.3 Processing

The system shall:

- Display the available items
- Add selected items to the order
- Process the payment
- Update the kitchen

3.3 Use Cases



3.3.1 Use Case #1- Purchase Tickets

Primary Actor: Customer

Description:
This use case describes the process of a customer purchasing one or more movie tickets through the FineLine Cinema Ticketing System.

Preconditions:

- The system is operational.
- The selected movie and showtime exist in the system.
- Tickets are available for purchase.

Main Flow:

1. The customer selects a theater location.
2. The customer selects a movie.
3. The customer selects a showtime.
4. The customer selects the number of tickets (maximum of 20).
5. If applicable, the customer selects ticket type (regular, student, military, senior).
6. The system displays a checkout page.
7. The system starts a five minute checkout timer.
8. The customer enters payment information.
9. The system processes payment through third party payment service.
10. The system confirms successful transactions.
11. The system generates digital tickets and receipts.
12. The system updates ticket availability in real time.

Alternative Flow:

- If payment fails, the system displays an error message and allows retry.
- If the checkout timer expires, selected tickets are released.

Postconditions:

- Ticket purchase is recorded in the database.
- Seat or ticket availability is updated.
- Confirmation notification is sent to the customer.

3.3.2 Use Case #2 – Select Seat (Deluxe Auditorium)

Primary Actor: Customer

Description:
This use case describes how a customer selects specific seats in a deluxe auditorium with assigned seating.

Preconditions:

- Deluxe auditorium selected.
- Seats are available for the selected showtime.

Main Flow:

1. The customer selects the theater location

2. The customer selects movies and showtimes.
3. The system displays an interactive seating chart.
4. The system shows seat status (available, selected, occupied).
5. The system identifies wheelchair accessible and companion seats.
6. The customer selects available seats.
7. The system temporarily reserves selected seats during the checkout timer.

Alternative Flow:

- If a selected seat becomes unavailable, the system displays a message and prompts reselection.
- If the checkout timer expires, reserved seats are released.

Postconditions:

- Selected seats are confirmed upon successful payment.
- Seat status is updated in the database.

3.4 Classes / Objects (recommended set)

3.4.1 Customer Account

CustomerAccount
- accountId : String **- membershipId : String** **- fullName : String** **- emailAddress : String** **- phoneNumber : String** **- passwordHash : String** **- storedPaymentMethods : List<PaymentMethod>** **- purchaseHistory : List<TicketOrder>**
+ AddPaymentMethod(String cardNumber, String cardName, String, String CVV, String ExpirationDate) : Void **+ CreateAccount(String fullName, String email, String password) : CustomerAccount** **+ UpdateAccount(String fullName, String email, String phoneNumber) : Void** **+ DeleteAccount() : Void** **+ Login() : Boolean** **+ Logout() : Void** **+ ViewOrders() : List<TicketOrder>**

References: Use Case 3.3.1 (Purchase Tickets)

3.4.2 Movie

Movie
- movieId : String **- title : String** **- genre : String** **- runtimeMinutes : Integer** **- rating : String** **- description : String** **- trailerURL : String** **- reviewScore : Float**
+ GetShowtimes(String theaterId) : List<Showtime> **+ GetDetails() : Movie**

References: Use Case 3.3.1 (Purchase Tickets)

3.4.3 Show Time

Showtime
- showTimeId : String **- movieId : String** **- theaterId : String** **- auditoriumId : String** **- startTime : DateTime** **- endTime : DateTime** **- isSoldOut : Boolean** **- ticketPriceRegular : Float** **- purchaseCutoffTime : DateTime**
+ IsPurchasable(DateTime currentTime) : Boolean **+ GetAvailability() : Integer** **+ UpdateShowtime(String adminId, DateTime startTime, DateTime endTime) : Void**

References: Use Case 3.3.1 (Purchase Tickets)

3.4.4 Seat

Seat
- theaterId : String **- auditoriumId : String** **- seatId : String** **- row : String** **- seatNumber : Integer** **- seatType : String // standard, wheelchair, companion** **- seatStatus : String // available, unavailable, selected**
+ Reserve(String orderId) : Boolean **+ ReleaseSeat() : Void** **+ MarkOccupied() : Void**

References: Use Case 3.3.2 (Select Seat)

3.4.5 Theater

Theater
- theaterId : String **- theaterAddress : String** **- numberOfAuditoriums : Integer // const**
+ Theater() : Theater **+ GetTheaterAddress() : String**

3.4.6 Auditorium

Auditorium
- auditoriumId : String **- theaterId : String** **- numberOfSeats : Integer** **- currentMovieId : Movie**
+ Auditorium() : Auditorium **+ ChangeMovie(Movie newMovie) : Void**

3.4.7 Ticket Order

TicketOrder
- orderId : String - accountId : String - showTimeId : String - ticketCount : Integer - ticketTypeCounts : Map<String, Integer> - selectedSeats: List<Seat> // deluxe only - ticketPrice : Float - localTaxes : Float - paymentStatus : String - checkoutStartTime : DateTime - confirmationCode : String - qrCode : String
+ CalculateTotal() : Float + StartCheckoutTimer() : Void + ConfirmPayment(String paymentToken) : Boolean + GenerateQRCode() : String + SendConfirmation() : Void

References: Use Case 3.3.1 (Purchase Tickets)

3.4.8 Food Order

FoodOrder
- ticketOrderId : String - orderStatus: String - orderType : String - allergies : String - orderPreparationTime : Integer
+ FoodOrder(List<String> items) : FoodOrder + CompleteOrder() : Void

3.5 Non-Functional Requirements

This section outlines the non-functional requirements for the FineLine Cinema Ticketing System: operational constraints, performance characteristics, and quality attributes. All requirements are testable and measurable.

3.5.1 Performance

- The system shall support at least 24,000 concurrent users during peak demand.
- Payment transactions shall be completed within 5 seconds
- processing at least 4,800 ticket purchase transactions per minute during peak demand
- support scaling to accommodate high-demand movie releases without system failure

payment two different systems; online and in person
The system shall support credit and debit card payments through a secure third-party payment provider.

3.5.2 Reliability

The system shall maintain reliable operation under normal and peak load conditions.

- The system shall achieve a Mean Time Between Failures (MTBF) greater than 30 days.
- In the event of a server failure, core system functionality shall be restored within 60 seconds.
- Automated database backups shall occur at intervals not exceeding 15 minutes.
- Backup data shall be stored securely and recoverable without data corruption.
- No completed transaction shall be lost due to system failure.

3.5.3 Availability

The system shall be accessible to users with minimal downtime.

- The system shall maintain at least 99% uptime per calendar month, excluding scheduled maintenance.
- The system shall be available 24 hours per day, 7 days per week, outside of scheduled maintenance windows.

- Scheduled maintenance shall not exceed 2 hours per week.
- In the event of infrastructure failure, automatic failover shall occur within 60 seconds.

3.5.4 Security

The system shall implement security mechanisms to protect user data, payment information, and system integrity.

Secure Communication

- All communication between client devices and the server shall use HTTPS with TLS encryption.
- No sensitive data shall be transmitted over unencrypted channels.

Payment Security

- The system shall comply with PCI-DSS standards for processing and storing payment information.
- Payment information shall not be permanently stored in the system database unless tokenized by the third-party payment provider.

Authentication and Authorization

- Users shall be required to authenticate using a valid username and password before accessing personal account information.
- The system shall enforce strong password requirements (minimum length, character complexity).
- Multi-factor authentication (MFA) shall be supported for administrator accounts.

Bot Protection

- The system shall implement CAPTCHA or equivalent human-verification mechanisms during checkout.
- The system shall enforce rate limiting to prevent automated ticket purchasing.

- The system shall monitor abnormal purchasing behavior and flag suspicious activity.

Data Protection

- User passwords shall be stored using secure hashing algorithms (e.g., bcrypt or Argon2).
- Sensitive user data shall be encrypted at rest.

3.5.5 Maintainability

The system shall be designed to allow efficient updates, bug fixes, and feature enhancements.

- The system shall follow a modular architecture in which components are logically separated to reduce interdependencies.
- Source code shall follow consistent coding standards and naming conventions.
- All major modules shall include inline documentation and API level documentation sufficient for future developers to understand system functionality.
- System logs shall be generated for critical events (e.g., failed payments, authentication attempts, system errors).
- System logs shall be retained for a minimum of 30 days.
- The system shall allow updates to movie listings, pricing, and showtimes without requiring system downtime.

3.5.6 Portability

The system shall be deployable and accessible across multiple environments and platforms.

- The system shall operate through modern web browsers, including Chrome, Safari, Microsoft Edge, and Firefox.
- The system shall be accessible on desktop computers, laptops, tablets, and mobile browsers.
- The system shall support deployment on Linux-based cloud server environments.
- The system shall not require the installation of additional client-side software.

3.6 Brief Overview of System

The FineLine Cinema Ticketing System is a web-based client-server application designed to support real-time ticket purchasing, seat management, food ordering, and administrative theater operations. The system follows a layered client-server architecture consisting of a presentation layer (web interface), an application logic layer (business processing), and a centralized database layer shared across all theater locations.

The system enables secure online transactions while maintaining real-time synchronization between online purchases and in-person ticket sales. All data is stored in a centralized database partitioned by theater location to ensure consistency, scalability, and high availability.

From a Customer Perspective

From a customer perspective, the system provides an intuitive web interface that allows users to:

- Browse currently playing and upcoming movies
- View showtimes by theater location
- Select seats (for deluxe auditoriums) using an interactive seating chart
- Purchase tickets securely through a third-party payment provider
- Order food and drinks during checkout or during the movie
- Receive digital tickets with QR codes for scanning at the theater
- Manage account information and view purchase history

Customers interact only with the presentation layer, while all business rules, validation, seat locking, and payment processing occur within the application layer.

From an Administrator's Perspective

From an administrator's perspective, the system provides a secure administrative interface that allows authorized theater staff to:

- Manage movie listings and trailers
- Create and modify showtimes
- Update ticket pricing and promotions
- Manage seat availability and auditorium configurations
- Monitor ticket sales and order records

Administrator actions update the centralized database in real time, ensuring synchronization across all theaters and preventing data inconsistencies.

3.7 Design Constraints

The system must operate within standard web-based infrastructure constraints. The application must support secure HTTPS communication and comply with industry security practices for password storage and payment processing. Integration with a third-party payment gateway imposes API interface constraints and requires adherence to external service response formats.

The system must be deployable on standard cloud or on-premises server environments and support modern web browsers. Hardware limitations primarily relate to database storage capacity and server concurrency limits.

3.8 Logical Database Requirements

The system uses a centralized relational database to store persistent data. Logical database requirements include:

- Unique identifiers for all primary entities (e.g., accountId, orderId).
- Referential integrity between related tables (e.g., TicketOrder to CustomerAccount).
- Support for transactional consistency during ticket checkout to prevent seat conflicts.
- Storage of structured data types, including strings, integers, floating point values, and date time objects.
- Secure handling of sensitive information such as hashed passwords.

The database must support concurrent access and maintain atomicity during payment confirmation workflows.

3.9 Other Requirements

The system must support scalability to handle peak ticket purchasing periods. Future extensions may include loyalty rewards programs, analytics reporting, and enhanced fraud detection mechanisms.

4. Software Architecture Overview

The FineLine Cinema Ticketing System is a web-based client–server application. Customers and administrators interact through a web UI (and optional mobile app). Requests are sent to an application server (API layer) that enforces business rules (ticket limits, checkout timer, seat holds, pricing), reads/writes to the database, and integrates with external services for payment

processing and notifications. The system is designed so that core ticketing logic runs on the server, while the client focuses on presentation and user input.

4.1 Major Components

- **Customer Web/Mobile UI**: browse movies, select theater/showtime, select seats (deluxe), purchase tickets, view confirmation/QR.

- **Administrator Web UI**: manage movies, showtimes, pricing, and seating availability.

- **Application Server (Ticketing API / Business Logic)**: validates requests, handles checkout timer, seat reservation rules, pricing rules, ticket purchase workflow.

- **Database (Persistent Storage)**: stores accounts, movies, theaters, showtimes, seats, orders, payment statuses, and confirmations.

- **Payment Gateway (3rd party)**: processes payments (Visa/Mastercard/other).

- **Email/Notification Service**: sends confirmations/receipts and reminders.

- **QR Code Generator**: generates scannable ticket QR codes.

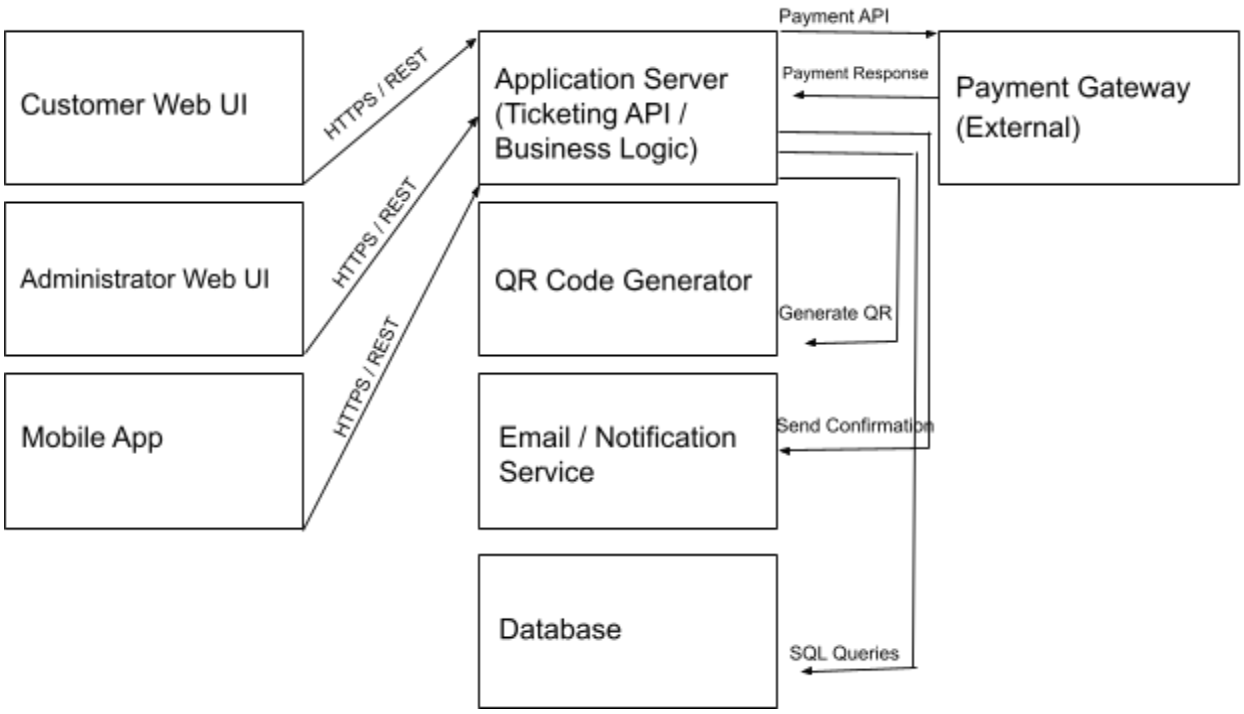
- *(Optional/extension)* **Bot Detection / Rate Limiting**: flags suspicious activity (high volume, abnormal speed).

4.2 Software Architecture Diagram

- The Software Architecture Diagram illustrates the high-level structure of the FineLine Cinema Ticketing System and the interactions between its major components.
- The Customer Web UI and Administrator Web UI communicate with the Application Server using secure HTTPS requests. The Application Server contains the core business logic and enforces ticketing rules, seat reservations, pricing, and checkout validation.
- The Application Server interacts with the centralized Database to store and retrieve persistent data, including user accounts, movies, showtimes, seats, and ticket orders.
- For payment processing, the Application Server communicates with an external Payment Gateway through a secure API. Upon successful transaction approval, the Application

Server generates a QR code using the QR Code Generator module and sends confirmation notifications through the Email/Notification Service.

- All external communications occur over encrypted channels to ensure system security and data protection.



4.3 UML Class Description

This section provides a detailed explanation of the UML Class Diagram for the FineLine Cinema Ticketing System. The diagram defines the core domain entities, their attributes, operations, and relationships. Each class represents a logical component of the system's business model and enforces specific responsibilities within the ticketing workflow.

CustomerAccount

The CustomerAccount class represents a registered user within the FineLine Cinema system. It manages user authentication, personal profile data, stored payment methods, and ticket purchase history. This class acts as the primary entry point for all customer-specific operations.

Attributes

- accountId : String
- membershipId : String (optional)
- fullName : String
- emailAddress : String
- phoneNumber : String (optional)
- passwordHash : String
- storedPaymentMethods : List<PaymentMethod>
- purchaseHistory : List<TicketOrder>

Operations

- createAccount(fullName:String, email:String, password:String) : CustomerAccount
- login(email:String, password:String) : Boolean
- logout() : Void
- updateProfile(fullName:String, email:String, phone:String) : Void
- addPaymentMethod(method:PaymentMethod) : Void
- viewOrders() : List<TicketOrder>

PaymentMethod

The PaymentMethod class represents customer-stored payment information used for transaction processing. It abstracts payment details while preserving security through partial masking and validation logic.

Attributes

- paymentMethodId : String
- accountId : String
- cardBrand : String
- last4 : String
- expMonth : Integer
- expYear : Integer
- billingZip : String

Operations

- validate() : Boolean
- mask() : String

TicketOrder

The TicketOrder class represents a customer's ticket purchase transaction. It coordinates seat selection, pricing calculations, checkout timing, payment confirmation, and digital ticket generation. This class governs the lifecycle of a ticket purchase from reservation to confirmation.

Attributes

- orderId : String
- accountId : String
- showTimeId : String
- ticketCount : Integer
- ticketTypeCounts : Map<String,Integer>
- selectedSeats : List<Seat>
- ticketPrice : Float
- localTaxes : Float
- paymentStatus : String
- checkoutStartTime : DateTime
- confirmationCode : String
- qrCode : String

Operations

- calculateTotal() : Float
- startCheckoutTimer() : Void
- confirmPayment(paymentToken:String) : Boolean
- generateQRCode() : String
- sendConfirmation() : Void

FoodOrder

The FoodOrder class models concession purchases associated with a ticket order. It manages item selection, preparation timing, allergy information, and order status tracking.

Attributes

- foodOrderId : String
- ticketOrderId : String
- items : List<String>
- orderStatus : String
- allergies : String
- orderPrepTimeMinutes : Integer

Operations

- createOrder(items:List<String>) : FoodOrder
- completeOrder() : Void

Movie

The Movie class represents films available for screening at one or more theaters. It contains descriptive metadata and provides access to associated showtimes.

Attributes

- movieId : String
- title : String
- genre : String
- runtimeMinutes : Integer
- rating : String
- description : String
- trailerURL : String
- reviewScore : Float

Operations

- getShowtimes(theaterId:String) : List<Showtime>
- getDetails() : Movie

Showtime

The Showtime class represents a scheduled screening of a movie within a specific auditorium. It manages availability logic, ticket pricing, and administrative updates.

Attributes

- showTimeId : String
- movieId : String
- theaterId : String
- auditoriumId : String
- startTime : DateTime
- endTime : DateTime

- isSoldOut : Boolean
- ticketPriceRegular : Float
- purchaseCutoffTime : DateTime

Operations

- isPurchasable(currentTime:DateTime) : Boolean
- getAvailability() : Integer
- updateShowtime(adminId:String, startTime:DateTime, endTime:DateTime) : Void

Theater

The Theater class represents a physical cinema location containing multiple auditoriums. It serves as a parent entity for Showtime and seating organization.

Attributes

- theaterId : String
- theaterAddress : String
- numberOfAuditoriums : Integer

Operations

- getTheaterAddress() : String

Auditorium

The Auditorium class represents an individual screening room within a theater. It maintains seat capacity and is associated with currently scheduled movies.

Attributes

- auditoriumId : String
- theaterId : String
- numberOfSeats : Integer
- currentMovieId : String

Operations

- changeMovie(newMovie:Movie) : Void

Seat

The Seat class represents an individual seat within an auditorium. It manages seat state transitions, including reservation, release, and final occupation after successful payment.

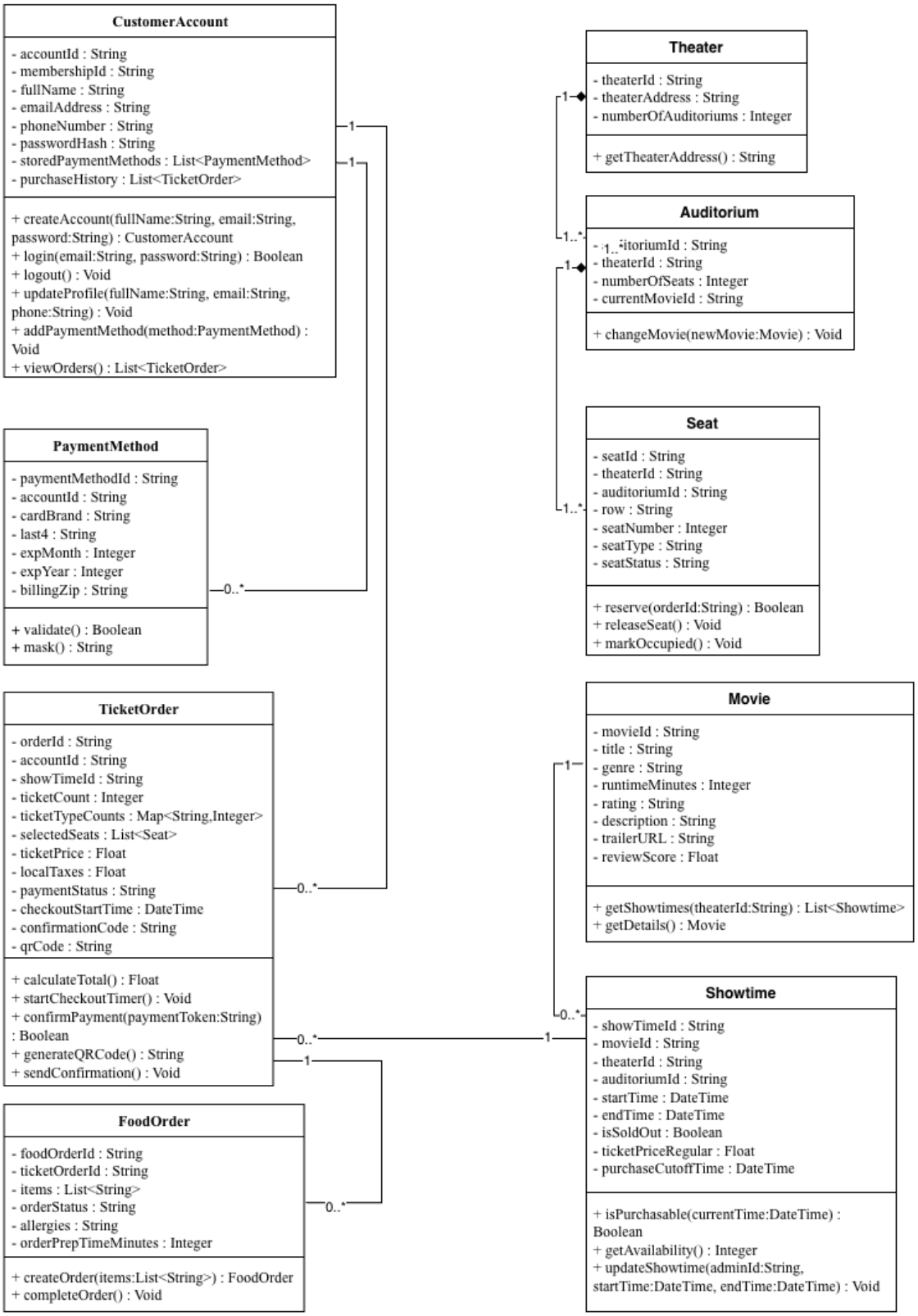
Attributes

- seatId : String
- theaterId : String
- auditoriumId : String
- row : String
- seatNumber : Integer
- seatType : String
- seatStatus : String

Operations

- reserve(orderId:String) : Boolean
- releaseSeat() : Void
- markOccupied() : Void

4.4 UML Class Diagram



5. Development Plan and Task Partitioning

5.1 System Implementation Breakdown

The FineLine Cinema Ticketing System will be implemented in modular subsystems corresponding to the functional requirements defined in the SRS. These subsystems include:

Core Backend Systems

- Account Management System
- Password Encryption System
- Database Query System
- Refund Management System
- Receipt Generator

Ticketing and Seat Systems

- Seat Selection System
- Seat Management System
- QR Code Ticket Generator
- Ticket Scanning System

Scheduling and Theater Systems

- Theater Selection System
- Show Time Management System
- Movie Selection System

Payment and Security Systems

- Payment Processing System
- Bot Detection System
- Age Verification System

Communication Systems

- Email Confirmation System
- Push Notification System

Client Interface Systems

- Website for Guests & Members
- Mobile Application
- Mobile Food Ordering System

5.2 Development Timeline

The project will be implemented over an 8-week development cycle.

Weeks 1–2: Database and Core Systems

- Implement Database Query System
- Implement Account Management and Password Encryption
- Implement Movie and Theater selection systems

Weeks 3–4: Ticketing & Scheduling

- Implement Show Time Management System
- Implement Seat Selection and Seat Management Systems
- Implement QR Code Ticket Generator

Week 5: Payment & Security Integration

- Integrate Payment Processing System with third-party API
- Implement Bot Detection System
- Implement Age Verification logic

Week 6: Communication Systems

- Implement Email Confirmation System
- Implement Push Notification System
- Implement Receipt Generator

Week 7: Client Applications

- Implement the website interface
- Implement Mobile Application
- Implement Mobile Food Ordering System

Week 8: Testing and Validation

- Integration testing across all subsystems
- Stress testing for concurrency
- Security validation
- Final documentation and GitHub review

5.3 Team Member Responsibilities

Each team member will be assigned primary ownership of specific subsystems:

Backend Lead

- Account Management
- Payment Processing
- Refund Management
- Bot Detection
- Database Query

Frontend Lead

- Website Interface
- Mobile Application
- Seat Selection UI
- Theater and Movie Selection UI

Systems & Integration Lead

- QR Code Generator
- Ticket Scanning System
- Email & Push Notifications
- Age Verification System

Database & Testing Lead

- Database schema design
- Referential integrity
- Stress testing
- Integration validation

All team members will contribute commits to the GitHub repository in compliance with project submission requirements.

GitHub Links

Esteban Naranjo

https://github.com/jaxun614/CS250-SRS/blob/main/FineLine_SRS_Software_Design_Specification.pdf

Jackson Adams

https://github.com/jaxun614/CS250-SRS/blob/main/software_requirenment_template.docx.pdf

Bryan Ly

6. Test Plan

6.1 Testing Overview

The purpose of this test plan is to verify and validate the functionality, reliability, and performance of the FineLine Cinema Ticketing System described in this Software Requirements Specification (SRS).

Testing ensures that the system operates in accordance with the functional and non-functional requirements defined in Sections 3 and 4 of this document. The testing process evaluates both individual components and the system as a whole.

Testing will be conducted using multiple levels of verification, including unit testing, integration testing, and system testing.

The test plan targets the following major components of the system architecture:

- Ticket purchasing workflow
- Seat selection and seat reservation logic
- Payment processing integration
- Database transaction handling

- System performance under high user demand

Testing will be performed using structured test sets and predefined test vectors to validate expected system behavior.

The detailed test cases for this test plan are documented in the Excel Test Case file in the project repository.

GitHub Repository Link: https://github.com/jaxun614/CS250-SRS

6.2 Updated Design Specification

The design specification for the FineLine Cinema Ticketing System remains consistent with the architecture defined in Assignment 2.

The system architecture includes the following primary components:

- Customer Web Interface
- Administrator Web Interface
- Application Server (Business Logic Layer)
- Centralized Database
- Third-Party Payment Gateway
- Email / Notification Service
- QR Code Generator

These components interact through HTTPS communication and internal API calls handled by the application server.

The UML Class Diagram presented in Section 4.2 represents the updated system design. No structural modifications to the class relationships were required for this testing plan. The diagram continues to represent the system entities, including:

- CustomerAccount
- PaymentMethod
- TicketOrder
- FoodOrder
- Movie
- Showtime
- Seat
- Theater
- Auditorium

Testing described in this section directly targets these components and verifies the behavior of the system according to the relationships defined in the UML Class Diagram.

6.2.1 Test Case Summary

A summary of representative test cases is provided below. Detailed test case specifications, including step by step procedures, inputs, expected outputs, and execution results, are documented in the Excel file (testCasesSample.xlsx) available in the project repository.

The Excel test case document includes:

- Test Case ID

- Component/Feature

- Test Description

- Pre-conditions

- Test Steps

- Expected Results

- Actual Results

- Status

- Assigned Tester

6.3 Unit Testing

Unit testing verifies the correctness of individual system components and class methods defined in the UML Class Diagram. Each unit test focuses on a single component to ensure that it performs its intended function independently of the rest of the system.

Unit Test Set 1 – Ticket Order Calculation

Description:
This unit test set verifies that the system correctly calculates the total price of a ticket order based on three factors: quantity, ticket type, and pricing. The ultimate goal is to have consistency within the pricing logic function before integration of the payment module.

Target Component:
TicketOrder class

Features Tested:

UT1.1 - Minimum Ticket Quantity

Input: 1 Standard Ticket.

Expected: One standard ticket and order is accepted.

Purpose: Verify that the system correctly processes the minimum valid ticket quantity.

UT1.2 - Validation of Ticket Quantity Limits

Input: 20 regular tickets

Expected: System calculates the correct total price and allows the order.

Purpose: Stress test, ensures that the system can handle the maximum allowed ticket quantity.

Test Vectors:

UT1.3 - Ticket Quantity Above Limit

Input: 21 tickets

Expected: System rejects the order, then displays an error indicating ticket quantity limit (20).

Purpose: Enforce ticket quantity and prevent invalid ticket orders.

UT1.4 - Mixed Ticket Types

Input: 2 standard tickets + 1 student ticket + 1 senior ticket

Expected: System calculates the correct total price.

Purpose: Verify that the system logic is correct and handles multiple ticket types.

UT1.1 - Standard Tickets

Input: 2 standard tickets ($12 per ticket)

Expected Result: total price $24

Purpose: Verify standard pricing logic.

UT1.2 – Deluxe Tickets

Input: 2 deluxe tickets ($18 per ticket)

Expected Result: total price $36

Purpose: Verify deluxe pricing logic.

UT1.3 - Mixed Ticket Order

Input: 2 standard tickets + 1 deluxe ticket

Expected Result: total price $42

Purpose: Verifies that the system correctly sums different ticket types.

UT1.4 - Zero Ticket Edge Case

Input: 0 tickets

Expected Result: total price $0

Purpose: Ensures that the system prevents invalid ticket orders

Expected Result:
The system correctly calculates the ticket total and prevents ticket purchases exceeding the allowed limit.

Assigned Team Member: Bryan Ly

Unit Test Set 2 – Seat Reservation Logic

Description:
This unit test set verifies that the seat reservation system correctly handles seat availability, conflicts, and reservation updates. Ensures that seats cannot be double-booked and that seat status is updated accordingly.

Target Component:
Seat class

Features Tested:

UT2.1 - Reserve Available Seat

Input: User selects available seating.

Expected: Seat status changes from available to reserved.

Purpose: Verify basic seat reservation functionality.

UT2.2 - Prevent Double Booking

Input: Two users attempt to book the same seat.

Expected: First reservation succeeds; second receives a message that the seat is unavailable.

Purpose: Ensures that the system prevents double booking

UT2.3 - Releasing seats when the checkout timer expires

Input: User selects seat(s), then cancels checkout before payment.

Expected: Seat returns to available status.

Purpose: Prevents overbooking in seats and is released when transactions are not completed.

Test Vectors:

UT2.4 - Seat Status Update

Input: User completes checkout with selected seats.

Expected: Seat status updates to occupied.

Purpose: Verify that seat status updates after the transaction is complete.

Expected Result:
The system correctly updates seat status and prevents duplicate seat reservations.

Assigned Team Member: Bryan Ly

6.4 Integration Testing

Integration testing verifies interactions between multiple components of the system architecture.

Integration Test Set 1 – Ticket Purchase and Database Update

Target Components: TicketOrder, Showtime, Seat, and Database system

Description:
This integration test set verifies that the ticket purchase workflow correctly updates the centralized database when a customer completes a valid ticket purchase. It also checks that seat availability is synchronized in real time and that duplicate seat bookings are prevented when multiple users interact with the same showtime.

Features Tested:

- Ticket purchase transaction processing
- Database update of seat availability
- Synchronization of reserved and occupied seat states
- Prevention of duplicate seat bookings during concurrent access

Test Vectors:

- Valid ticket purchase request for an available seat
- Multiple users attempt to select the same seat simultaneously
- Successful payment followed by seat status update
- Seat reservation conflict during concurrent checkout attempts

Expected Result:
The database correctly records the completed ticket purchase, updates seat availability in real time, and prevents duplicate seat bookings when multiple users attempt to reserve the same seat.

Assigned Team Member: Esteban Naranjo

Integration Test Set 2 – Payment Processing Integration

Target Components:
Application Server, Payment Gateway, and TicketOrder system

Description:
This integration test set verifies that the payment processing workflow correctly communicates with the external payment gateway and updates the system after a transaction is completed. It ensures that successful payments generate ticket confirmations and QR codes, while failed payments do not create ticket orders or reserve seats.

Features Tested:

- Payment authorization through the payment gateway
- Transaction confirmation and ticket order creation
- Handling of failed payment transactions

Test Vectors:

- Successful payment transaction using valid payment credentials
- Failed payment transaction due to declined or invalid payment information

Expected Result:
Successful payments result in ticket confirmation and QR code generation. Failed payments return an error message without creating an order record or reserving seats.

Assigned Team Member: Esteban Naranjo

6.5 System Testing

System testing evaluates the behavior of the entire system when all components operate together.

System Test Set 1 – End-to-End Ticket Purchase

Target Components:
Customer Web Interface, Application Server, Payment Gateway, TicketOrder system, and Database

Description:
This system test verifies that the complete ticket purchasing workflow functions correctly when all system components interact together. The test simulates a real customer browsing available movies, selecting a showtime and seats, completing payment, and receiving confirmation of the ticket purchase.

Features Tested:

- Movie browsing and showtime selection
- Seat selection and reservation handling
- Payment processing and authorization
- Ticket order creation and confirmation generation

Test Vectors:

- Complete ticket purchase scenario performed by a customer
- Valid payment transaction for selected seats

- Generation of confirmation message and QR code ticket

Expected Result:
The system successfully processes the entire purchase transaction, reserves the selected seats, generates a digital ticket confirmation with a QR code, and updates the database accordingly.

Assigned Team Member: Esteban Naranjo

System Test Set 2 – High Concurrency Load Test

Target Components:
Application Server, Database System, and Seat Reservation Module

Description:
This system test evaluates the ability of the FineLine Cinema Ticketing System to handle a large number of simultaneous users accessing the platform. The test simulates high traffic conditions to ensure the system maintains performance stability, properly processes ticket purchases, and prevents seat reservation conflicts during periods of heavy demand.

Features Tested:

- System stability under heavy user load
- Concurrent seat reservation handling
- Database consistency during multiple simultaneous transactions
- Server response time under high traffic conditions

Test Vectors:

- Simulated system access by 10,000 concurrent users attempting ticket purchases
- Increased load scenario with up to 20,000 concurrent users performing seat selection and checkout
- Multiple users are attempting to reserve the same seat simultaneously

Expected Result:
The system maintains stable performance under high load conditions, processes transactions without crashing or slowing significantly, and prevents duplicate seat bookings or database inconsistencies.

Assigned Team Member: Esteban Naranjo

System Test Set 3 – System Restart and Downtime Test

Target Components: Application Server, Database, and Failover Infrastructure

Features Tested:

- Automatic failover under infrastructure failure
- Data integrity during an unexpected system restart
- Seat reservation handling during downtime
- Recovery time within the required 60-second threshold

Test Vectors:

- Simulated server failure during an active customer checkout session
- Forced system restart while concurrent users are mid-transaction
- Recovery verification after simulated infrastructure failure

Expected Result:
The system restores core functionality within 60 seconds of failure. No completed transactions are lost or duplicated. Seats that were mid-reservation during the failure are either properly held or released back to available status. Users mid-checkout receive an appropriate error message and can reattempt their purchase without data conflicts

Assigned Team Member: Jackson Adams

6.6 Test Case Documentation

A set of **12 detailed test cases** is provided in the Excel file:

testCasesSample.xlsx

The Excel file includes:

- Test Case ID
- Test Description
- Target Feature
- Input Test Data
- Expected Output
- Pass/Fail Criteria
- Tester Name

These test cases correspond to the unit, integration, and system testing sets described in this section.